Action Plus Sports Images — Deployment Guide

Overview

Static website for Action Plus Sports Images (actionplus.co.uk) that displays a responsive gallery grid of recent sporting event photography. Gallery thumbnails and metadata are synced from PhotoShelter via a scheduled script.

GitHub repo: https://github.com/bkupbens/apsi

Technology Stack

- **Frontend:** Static HTML, CSS, vanilla JavaScript (no frameworks, no build step)
- **Sync scripts:** Two options available (see below)
- **Image source:** PhotoShelter (actionplusps.photoshelter.com)
- **No database required**

Sync Script Options

Script	Requirements	Best for
`sync.sh` (Bash)	bash 3+, curl, awk, sed, ImageMagick 6.7+	Production hosting, servers without Python 3
`serve.py --sync` (Python)	Python 3.10+, Pillow	Local development, hosts with Python 3

Both scripts produce identical output — the same `galleries.json` and WebP thumbnail images.

File Structure

```
apsi/
├── index.html              # Home page with photo gallery grid
├── about.html              # About page
├── photographers.html      # Prospective photographers page
├── contact.html            # Contact page
├── terms.html              # Terms & conditions
├── privacy.html            # Privacy policy
├── 404.html                # Custom 404 page
├── css/
│   ├── style.css           # All styles
│   └── fonts/              # Self-hosted WOFF2 fonts (Bebas Neue, DM Sans)
├── js/
│   └── app.js              # Gallery grid logic, search, navigation
├── images/                 # Gallery thumbnail images (WebP, ~100 files)
│   └── logo.png            # Site logo
├── galleries.json          # Gallery manifest (names, URLs, image paths, counts)
├── etags.json              # ETag cache for change detection (auto-generated)
```

```
├── sync.sh                 # Bash sync script (recommended for production)
├── serve.py                # Python sync script + local dev server
├── crontab.example         # Example cron schedule for automated sync
├── .htaccess               # Apache rewrite rules (if using Apache)
├── .gitignore
├── robots.txt
├── sitemap.xml
├── favicon.ico
├── favicon-32x32.png
└── apple-touch-icon.png
```

How It Works

1. **Static site** — All HTML pages are plain static files. No server-side rendering required.
2. **Gallery grid** — `index.html` loads `galleries.json` via JavaScript fetch and renders a responsive photo grid. Each cell links to the corresponding PhotoShelter gallery page.
3. **Sync script** — Scrapes the PhotoShelter gallery-list page, downloads cover images, converts them to WebP thumbnails (1200px wide, quality 84), and writes `galleries.json`. Uses ETags to detect changed covers and cleans up removed galleries.
4. **Image count** — The sync also scrapes the number of images per gallery from PhotoShelter and includes it in `galleries.json`, displayed as a badge on each grid cell.

Hosting Requirements

Minimum (Static Demo)

- Any static file host (GitHub Pages, Netlify, Cloudflare Pages, S3, Apache, Nginx)
- No server-side language needed
- Gallery images won't auto-update without the sync script

Production (Auto-Updating with Bash)

- Static file hosting for the site
- **bash 3+** (standard on virtually all Linux/Unix servers)
- **curl** (for HTTP requests)
- **ImageMagick 6.7+** (for JPEG to WebP conversion via `convert`)
- Cron job to run the sync periodically
- Write access to the `images/` directory, `galleries.json`, and `etags.json`

Alternative: Production with Python

- Python 3.10+
- **Pillow** (`pip install Pillow`)
- Cron job to run the sync periodically
- Write access to the `images/` directory, `galleries.json`, and `etags.json`

Setup Instructions

1. Clone the repository

```
git clone https://github.com/bkupbens/apsi.git
cd apsi
```

2. Run initial sync

Using Bash (recommended for production):

```
bash sync.sh
```

Using Python (alternative):

```
pip install Pillow
python3 serve.py --sync
```

Both will: - Fetch the gallery list from PhotoShelter - Download ~100 cover images as WebP to `images/` - Generate `galleries.json` with gallery metadata and image counts - Generate `etags.json` for change tracking

3. Configure scheduled sync

Add a cron job to run the sync periodically (e.g., every hour):

Bash version:

```
23 * * * * cd /path/to/apsi && bash sync.sh >> /tmp/apsi-sync.log 2>&1
```

Python version:

```
23 * * * * cd /path/to/apsi && python3 serve.py --sync >> /tmp/apsi-sync.log 2>&1
```

See `crontab.example` for reference.

4. Serve the site

Point your web server document root to the `apsi/` directory. No special server-side configuration is needed beyond serving static files.

For local development:

```
python3 serve.py
# Serves on http://localhost:8000
```

Key Configuration

Setting	Location	Value
Max galleries displayed	`js/app.js` line 10	`MAX_GALLERIES = 60`
Grid column width	`js/app.js` line 56	`Math.floor(W / 360)`
Image quality	`sync.sh` / `serve.py`	`quality 84` (WebP)
Image width	`sync.sh` / `serve.py`	`1200px` (from PhotoShelter)
PhotoShelter URL	`sync.sh` / `serve.py`	`actionplusps.photoshelter.com`

Apache Configuration

If using Apache, the included `.htaccess` file handles: - Custom 404 error page - Cache headers for static assets

Notes

- All fonts are self-hosted in `css/fonts/` (GDPR compliant — no external font requests).
- The site makes no tracking cookies or analytics calls.
- The `images/` directory is currently tracked in git to support the GitHub Pages demo. For production, you may want to add it back to `.gitignore` and generate images on the server via the sync script.
- The `WORDPRESS-PLUGIN-SPEC.md` file is a reference document from an earlier project phase and is not part of the live site.